endpoints are used by the routing and endpoint middleware. You'll learn more about endpoints and routing in chapter 5.

TIP You can define the endpoints for your app by using `MapGet()` anywhere in Program.cs before the call to `app.Run()`, but the calls are typically placed after the middleware pipeline definition.

In chapter 3, I mentioned that `WebApplication` automatically adds middleware to your app. You can see this process in action in listing 4.3 automatically adding the `EndpointMiddleware` to the end of the middleware pipeline. `WebApplication` also automatically adds the developer exception page middleware to the *start* of the middleware pipeline when you're running in development. As a result, you can omit the call to `UseDeveloperExceptionPage()` from listing 4.3, and your middleware pipeline will be essentially the same.

WebApplication and autoadded middleware

`WebApplication` and `WebApplicationBuilder` were introduced in .NET 6 to try to reduce the amount of boilerplate code required for a `Hello World!` ASP.NET Core application. As part of this initiative, Microsoft chose to have `WebApplication` *automatically* add various middleware to the pipeline. This decision alleviates some of the common getting-started pain points of middleware ordering by ensuring that, for example, `UseRouting()` is always called before `UseAuthorization()`.

Everything has trade-offs, of course, and for `WebApplication` the trade-off is that it's harder to understand exactly what's in your middleware pipeline without having deep knowledge of the framework code itself.

Luckily, you don't need to worry about the middleware that `WebApplication` adds for the most part. If you're new to ASP.NET Core, generally you can accept that `WebApplication` will add the middleware only when it's necessary and safe to do so.